Repository Katmanı Rehberi

Spring Data JPA ile veri erişim katmanını temelden öğrenmek

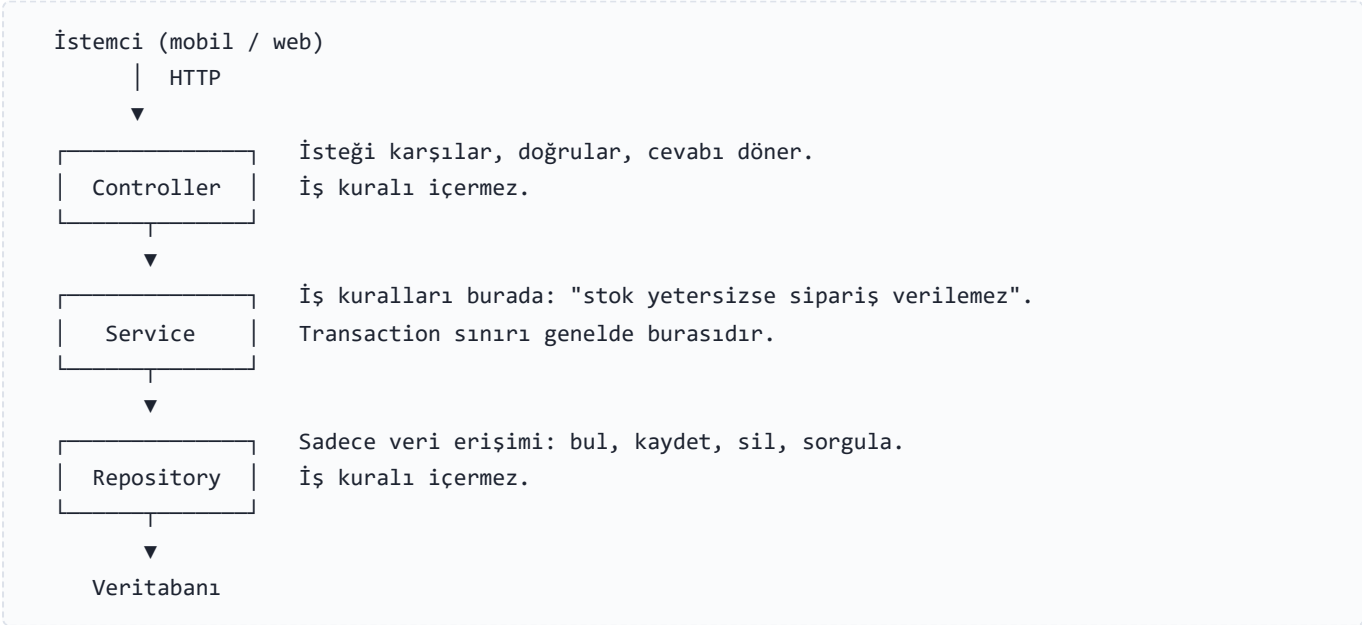
Bu rehber belirli bir projeye bağlı değildir. Örneklerde herkesin tanıdığı basit bir alan kullanılır: **Customer** (müşteri) ve **Order** (sipariş). Bir müşterinin birden çok siparişi olabilir.

1. Repository katmanı nedir?
2. Teknoloji katmanları (detaylı): SQL, JDBC, ORM, JPA, Hibernate, JPQL, HQL, Criteria, native query, Spring Data JPA
3. Örnek entity'ler
4. Repository arayüz hiyerarşisi
5. Hazır gelen metotlar
6. Metot adından sorgu (derived query)
7. @Query ile JPQL
8. Native query
9. Veri değiştiren sorgular: @Modifying
10. Dönüş tipleri
11. Sayfalama ve sıralama
12. Projection: sadece gereken alanlar
13. Dinamik sorgular: Specification
14. Transaction yönetimi
15. Persistence context ve entity yaşam döngüsü
16. Performans: N+1, LAZY/EAGER, fetch join
17. Kilitleme: optimistic ve pessimistic
18. Özel repository implementasyonu
19. Repository testleri
20. Sık yapılan hatalar
21. Kopya kâğıdı
22. Terimler sözlüğü

---

1. Repository katmanı nedir?

Repository, uygulamanın veriye erişen tek kapısıdır. Uygulamanın geri kalanı verinin nerede ve nasıl saklandığını bilmez; sadece "bana şu müşteriyi getir", "bu siparişi kaydet" der.



Neden ayrı bir katman?

- **Sorumlulukların ayrılması:** Service "ne yapılacağını", repository "verinin nasıl getirileceğini" bilir. Biri değişince diğeri etkilenmez.
- **Test edilebilirlik:** Service'i test ederken repository'yi sahte (mock) bir nesneyle değiştirebilirsin, gerçek veritabanına ihtiyaç kalmaz.
- **Tekrar kullanım:** "Aktif müşterileri getir" sorgusu bir kez yazılır, her yerden çağrılır.
- **Değiştirilebilirlik:** Veritabanı veya sorgu yöntemi değişse bile service kodu aynı kalır.

Repository ve DAO farkı

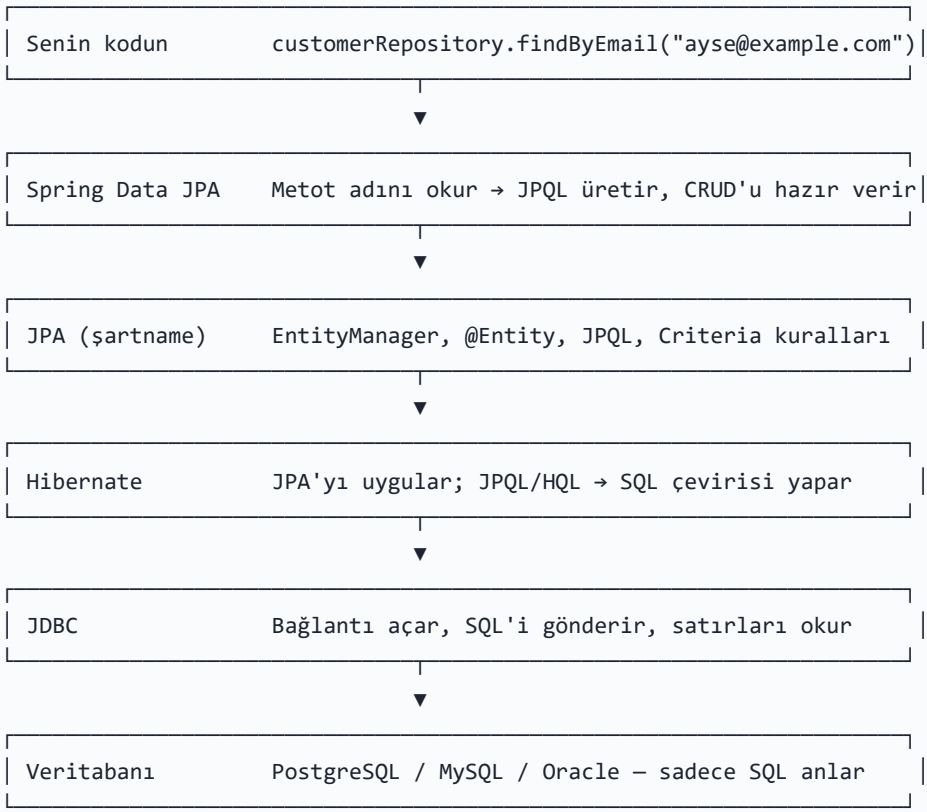
	DAO (Data Access Object)	Repository
Bakış açısı	Tabloya yakın: "bu tabloya şu işlemleri yap"	Alana (domain) yakın: "müşteri koleksiyonu gibi davran"
Tipik metotlar	`insert`, `update`, `selectById`	`save`, `findById`, `findActiveCustomers`
Köken	Klasik Java EE	Domain-Driven Design (Eric Evans)

Pratikte Spring dünyasında ikisi neredeyse aynı anlamda kullanılır. Spring Data'nın arayüzleri "Repository" adını taşır.

> **Altın kural**
> Repository'de **iş kuralı olmaz**. "Müşteri banlıysa sipariş verme" gibi bir kontrol service'e aittir. Repository sadece veriyi getirir ve saklar.

2. Teknoloji katmanları: SQL'den Spring Data JPA'ya

Repository yazarken sürekli birbirine benzeyen isimlerle karşılaşırsın: SQL, JDBC, JPA, Hibernate, JPQL, HQL, Criteria, native query, Spring Data JPA. Bunlar birbirinin alternatifi değil; çoğu **üst üste duran katmanlardır**. Her katman bir alttakinin zahmetli işini üstlenir.



Sorgu dilleri ise bu katmanların **içinde** konuşulan dillerdir:

Dil	Hangi katmanın dili?	Neyle yazılır?
SQL	Veritabanı	Tablo ve kolon adları
JPQL	JPA	Entity ve alan adları
HQL	Hibernate	Entity ve alan adları (JPQL + ekstralar)
Criteria API	JPA	Java kodu (metot çağrıları)
Native query	JPA üzerinden doğrudan veritabanı	Düz SQL

2.1 SQL — veritabanının dili

SQL (Structured Query Language), ilişkisel veritabanlarının anladığı tek dildir. Ne kadar katman kullanırsan kullan, en sonunda veritabanına giden **her zaman SQL'dir**. Diğer her şey SQL üretmenin farklı yollarıdır.

İki ana grup

DDL — yapıyı tanımlar (Data Definition)

DML — veriyi işler (Data Manipulation)

```
CREATE TABLE customers (
  id         BIGSERIAL PRIMARY KEY,
  first_name VARCHAR(50)  NOT NULL,
  email      VARCHAR(100) NOT NULL UNIQUE,
  status     VARCHAR(20)  NOT NULL,
  created_at TIMESTAMP    NOT NULL
);
ALTER TABLE customers ADD phone VARCHAR(20);
DROP TABLE customers;
```

```
INSERT INTO customers (first_name, email, status,
created_at)
VALUES ('Ayşe', 'ayse@example.com', 'ACTIVE',
now());

SELECT id, first_name FROM customers
WHERE status = 'ACTIVE' ORDER BY first_name;

UPDATE customers SET status = 'PASSIVE' WHERE id
= 7;

DELETE FROM customers WHERE id = 7;
```

SQL'in özellikleri

- **Tablo ve kolon adlarıyla** çalışır: `customers`, `first_name`. Java sınıflarından haberi yoktur.
- **Sonuç satırlar ve kolonlardır**, nesne değil. "Bir müşteri ve siparişleri" gibi iç içe yapılar doğrudan gelmez; JOIN ile düz bir tablo gelir.
- **Veritabanları arasında farklılık (lehçe / dialect) vardır.** Temel komutlar aynıdır ama ayrıntılar değişir:

İhtiyaç	PostgreSQL	MySQL	Oracle / SQL Server
İlk 10 kayıt	`LIMIT 10`	`LIMIT 10`	`FETCH FIRST 10 ROWS ONLY`
Büyük/küçük harf duyarsız arama	`ILIKE`	`LIKE` (varsayılan duyarsız)	`UPPER(x) LIKE UPPER(?)`
Otomatik artan ID	`BIGSERIAL` / `IDENTITY`	`AUTO_INCREMENT`	`IDENTITY` / `SEQUENCE`
Şu anki zaman	`now()`	`NOW()`	`SYSDATE` / `GETDATE()`

Bu farklılıklar, üst katmanların (JPQL, Hibernate) neden var olduğunun sebeplerinden biridir: sen bir kez yazarsın, doğru lehçeye çeviri onların işidir.

Pratikte

Spring Boot'ta `spring.jpa.show-sql=true` (ve okunaklı olsun diye `spring.jpa.properties.hibernate.format_sql=true`) açarsan, üst katmanlarda ne yazarsan yaz veritabanına giden gerçek SQL'i konsolda görürsün. Öğrenirken en faydalı ayardır.

2.2 JDBC — Java ile veritabanı arasındaki kablo

JDBC (Java Database Connectivity), Java'nın veritabanına bağlanıp SQL göndermesini sağlayan en alt seviye standart API'dir (`java.sql` paketi). Her veritabanı kendi **JDBC sürücüsünü** (driver) sağlar; örneğin PostgreSQL için `org.postgresql.Driver`. Üstteki bütün katmanlar (Hibernate dahil) en sonunda JDBC'yi kullanır.

Elle JDBC ile bir müşteri bulmak

```
public Optional<Customer> findByEmail(String email) {
    String sql = "SELECT id, first_name, email, status, created_at FROM customers WHERE email = ?";

    try (Connection conn = dataSource.getConnection();          // 1) bağlantı al
         PreparedStatement ps = conn.prepareStatement(sql)) {    // 2) sorguyu hazırla

        ps.setString(1, email);                                  // 3) parametreyi bağla

        try (ResultSet rs = ps.executeQuery()) {                 // 4) çalıştır
            if (!rs.next()) return Optional.empty();

            Customer c = new Customer();                          // 5) satırı nesneye ELLE çevir
            c.setId(rs.getLong("id"));
            c.setFirstName(rs.getString("first_name"));
            c.setEmail(rs.getString("email"));
            c.setStatus(CustomerStatus.valueOf(rs.getString("status")));
            c.setCreatedAt(rs.getTimestamp("created_at").toLocalDateTime());
            return Optional.of(c);
        }
    } catch (SQLException e) {                                    // 6) checked exception
        throw new RuntimeException(e);
    }
}
```

Temel JDBC nesneleri

Nesne	Görevi
`DataSource`	Bağlantı fabrikası. Spring Boot'ta varsayılan olarak **HikariCP** bağlantı havuzudur (her istekte yeni bağlantı açmak yerine hazır bağlantıları tekrar kullanır).
`Connection`	Veritabanıyla açık bir oturum. Transaction'lar bunun üzerinde yürür.
`PreparedStatement`	`?` yer tutuculu, önceden derlenmiş sorgu. Parametreler ayrı gönderildiği için **SQL injection'a karşı güvenlidir**.
`ResultSet`	Sorgu sonucu; satır satır gezilen bir imleç.

JDBC'nin zahmetleri — üst katmanların çözdüğü sorunlar

- **Tekrar eden kod:** Her sorguda bağlantı aç, hazırla, çalıştır, kapat.
- **Elle eşleme:** Her kolonu tek tek nesne alanına kopyalamak. 20 alanlı bir tabloda 20 satır.
- **İlişkiler:** Müşterinin siparişlerini getirmek için ayrı sorgu ve ayrı eşleme kodu.
- **Kaynak yönetimi:** Bağlantıyı kapatmayı unutursan havuz tükenir ve uygulama kilitlenir.
- **Lehçe bağımlılığı:** SQL'i elle yazdığın için veritabanı değişirse sorgular da değişir.

Ara basamak: Spring JdbcTemplate

Spring, JDBC'nin tekrar eden kısımlarını (bağlantı açma/kapama, exception çevirme) üstlenen `JdbcTemplate` sınıfını sunar. Yine SQL yazarsın ama kod kısalır:

```
Customer c = jdbcTemplate.queryForObject(
    "SELECT * FROM customers WHERE email = ?",
    (rs, rowNum) -> mapRow(rs),   // eşlemeyi yine sen yazarsın
    email);
```

ORM istemeyen, SQL üzerinde tam kontrol isteyen projelerde hâlâ çok kullanılır.

2.3 ORM fikri — nesneler ile tablolar arasındaki köprü

Java dünyası **nesnelerle**, veritabanı dünyası **tablolarla** düşünür. İkisi birbirine tam oturmaz; buna **object-relational impedance mismatch** (nesne-ilişkisel uyumsuzluk) denir.

Konu	Java (nesne dünyası)	Veritabanı (ilişkisel dünya)
Yapı	Sınıf, alan	Tablo, kolon
İlişki	Referans: `order.getCustomer()`	Yabancı anahtar: `customer_id`
Koleksiyon	`List<Order> orders`	Yok; diğer tabloda `customer_id` ile filtrelenir
Kimlik	`==` veya `equals()`	Primary key
Kalıtım	`class Admin extends User`	Doğrudan karşılığı yok
Tipler	`enum`, `LocalDateTime`, `BigDecimal`	`VARCHAR`, `TIMESTAMP`, `NUMERIC`

ORM (Object-Relational Mapping) bu iki dünya arasında otomatik çeviri yapar. Sen sınıfına "bu sınıf şu tabloya, bu alan şu kolona karşılık gelir" diye **bir kez** anlatırsın; sonrasında nesnelerle çalışırsın, SQL'i ORM üretir.

ORM'siz (JDBC) — kaydetme

```
ps = conn.prepareStatement(
  "INSERT INTO customers (first_name, email,
status, created_at) VALUES (?, ?, ?, ?)");
ps.setString(1, c.getFirstName());
ps.setString(2, c.getEmail());
ps.setString(3, c.getStatus().name());
ps.setTimestamp(4,
Timestamp.valueOf(c.getCreatedAt()));
ps.executeUpdate();
```

ORM ile — kaydetme

```
entityManager.persist(customer);
// INSERT sorgusunu ORM üretir,
// üretilen id'yi nesneye yazar.
```

2.4 JPA — ORM'in kural kitabı

JPA (Jakarta Persistence API), Java'da bir ORM'in nasıl davranması gerektiğini tanımlayan **resmi şartnamedir**. Bir kütüphane değil, bir sözleşmedir: arayüzler, anotasyonlar ve kurallardan oluşur. Tek başına hiçbir sorgu çalıştırmaz; onu uygulayan bir motora (Hibernate gibi) ihtiyaç duyar.

> **Benzetme**
> JPA, "bir arabanın direksiyonu, gaz ve fren pedalı şu şekilde olmalı" diyen bir **standart** gibidir. Hibernate ve EclipseLink bu standarda uyan farklı **arabalardır**. Sürmeyi bir kez öğrenirsen (JPA), hangi arabaya binersen bin kullanırsın.

JPA neleri tanımlar?

Parça	Örnek
Eşleme anotasyonları	`@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@ManyToOne`, `@OneToMany`, `@JoinColumn`, `@Enumerated`, `@Version`
EntityManager API	`persist`, `find`, `merge`, `remove`, `createQuery`, `flush`
Sorgu dili	JPQL
Programatik sorgu	Criteria API
Yaşam döngüsü	Transient, managed, detached, removed durumları; `@PrePersist`, `@PreUpdate` gibi olaylar
Kilitleme ve transaction	`LockModeType`, `@Version`

Saf JPA ile çalışmak: EntityManager

Spring Data olmadan, doğrudan JPA'nın `EntityManager` arayüzüyle de çalışabilirsin. Spring Data JPA'nın arka planda yaptığı tam olarak budur:

```
@Repository
public class CustomerDao {

    @PersistenceContext
    private EntityManager em;

    public void save(Customer c)          { em.persist(c); }                      // INSERT
    public Customer find(Long id)         { return em.find(Customer.class, id); }   // SELECT ... WHERE
id = ?
    public Customer update(Customer c)    { return em.merge(c); }                 // detached → managed
    public void delete(Customer c)        { em.remove(em.contains(c) ? c : em.merge(c)); }

    public List<Customer> findActive() {
        return em.createQuery("SELECT c FROM Customer c WHERE c.status = :s", Customer.class)
                 .setParameter("s", CustomerStatus.ACTIVE)
                 .getResultList();
    }
}
```

javax mı, jakarta mı?

JPA eskiden `javax.persistence` paketindeydi (Java EE). Oracle, Java EE'yi Eclipse Vakfı'na devredince isim **Jakarta EE** oldu ve paket `jakarta.persistence`'a taşındı. Spring Boot 3 ve sonrası yalnızca `jakarta.*` kullanır. İnternette `javax.persistence` içeren örnek görürsen eski bir sürüme aittir; import'u `jakarta` ile değiştirmen yeterlidir.

2.5 Hibernate — JPA'yı çalıştıran motor

Hibernate, JPA şartnamesinin en yaygın **uygulamasıdır (implementation)**. JPA'daki arayüzlerin arkasındaki gerçek kodu o yazar. Spring Boot'a `spring-boot-starter-data-jpa` eklediğinde Hibernate varsayılan olarak gelir. Hatta JPA'dan önce vardı; JPA standardı büyük ölçüde Hibernate örnek alınarak yazıldı.

Hibernate'in arka planda yaptıkları

Görev	Açıklama
SQL üretmek	`persist`, `find`, JPQL/HQL sorgularını doğru SQL'e çevirir.
Lehçe (dialect)	Hangi veritabanına bağlıysa onun lehçesinde SQL üretir (`PostgreSQLDialect`, `MySQLDialect` ...). Spring Boot bunu genelde otomatik algılar.
Şema yönetimi	`ddl-auto` ayarıyla entity'lerden tablo oluşturabilir veya kontrol edebilir.
Satır → nesne	`ResultSet`'i okuyup entity nesneleri oluşturur ("hydration").
Persistence context	Yüklenen nesneleri transaction boyunca hafızada tutar (birinci seviye önbellek).
Dirty checking	Managed nesnede değişen alanları fark edip commit'te UPDATE atar.
Lazy loading	LAZY ilişkiler için sahte (proxy) nesneler üretir; ilk erişimde SELECT atar.

Örnek 1 — Entity'den tablo üretimi

Yazdığın entity

```java
@Entity @Table(name = "customers")
public class Customer {
  @Id
  @GeneratedValue(strategy = IDENTITY)
  Long id;

  @Column(nullable = false, length = 50)
  String firstName;

  @Column(nullable = false, unique = true)
  String email;

  @Enumerated(EnumType.STRING)
  CustomerStatus status;
}
```

Hibernate'in ürettiği DDL (PostgreSQL, yaklaşık)

```sql
create table customers (
  id bigint generated by default
     as identity,
  first_name varchar(50) not null,
  email varchar(255) not null,
  status varchar(255) check (status in
     ('ACTIVE','PASSIVE','BANNED')),
  primary key (id)
);
alter table customers
  add constraint uk_... unique (email);
```

`firstName` alanının `first_name` kolonuna dönüşmesini Spring Boot'un varsayılan isimlendirme stratejisi (camelCase → snake_case) sağlar.

Örnek 2 — Aynı sorgu, farklı lehçeler

```java
// Senin yazdığın (JPQL + sayfalama)
em.createQuery("SELECT c FROM Customer c ORDER BY c.firstName", Customer.class)
  .setFirstResult(20).setMaxResults(10).getResultList();
```

Veritabanı	Hibernate'in ürettiği SQL'in sonu
PostgreSQL / MySQL	`... order by c.first_name limit ? offset ?`
Oracle 12c+ / SQL Server	`... order by c.first_name offset ? rows fetch next ? rows only`

Sen kodu değiştirmezsin; veritabanı değişince sadece lehçe değişir.

Örnek 3 — ddl-auto seçenekleri

Değer	Davranış	Nerede?
`none`	Şemaya hiç dokunmaz	Canlı ortam
`validate`	Tablolar entity'lerle uyumlu mu kontrol eder, uyumsuzsa uygulama açılmaz	Canlı ortam (Flyway/Liquibase ile birlikte)
`update`	Eksik tablo ve kolonları ekler; asla silmez veya tip değiştirmez	Geliştirme
`create`	Her açılışta tabloları silip yeniden oluşturur, **veri gider**	Deneme
`create-drop`	`create` ve kapanışta tabloları siler	Testler

Hibernate'e özel anotasyonlar

`@CreationTimestamp`, `@UpdateTimestamp`, `@BatchSize`, `@Formula` gibi anotasyonlar `org.hibernate.annotations` paketindedir; JPA standardının parçası değildir. Çok kullanışlıdırlar, ama bunları kullandığında kodun Hibernate'e bağlanır. Pratikte projelerin neredeyse hepsi Hibernate kullandığı için bu genelde sorun olmaz; yine de farkında olmak gerekir.

2.6 JPQL — nesnelerle konuşan sorgu dili

JPQL (Jakarta Persistence Query Language), JPA'nın sorgu dilidir. Söz dizimi SQL'e çok benzer ama arada temel bir fark vardır: **tablolar değil entity'ler, kolonlar değil Java alanları** sorgulanır. Hibernate JPQL'i alıp bağlı olduğu veritabanının SQL'ine çevirir.

Yan yana: JPQL ve SQL

JPQL (senin yazdığın)

```
SELECT c
FROM Customer c
WHERE c.status = :status
  AND c.createdAt >= :since
ORDER BY c.firstName
```

SQL (Hibernate'in ürettiği)

```
select c.id, c.created_at, c.email,
       c.first_name, c.status
from customers c
where c.status = ?
  and c.created_at >= ?
order by c.first_name
```

Kural	Açıklama
Entity adı	`Customer` (sınıf adı), tablo adı `customers` değil. Entity adı **büyük/küçük harfe duyarlıdır**.
Alan adı	`c.firstName`, kolon adı `first_name` değil. Alan adları da duyarlıdır.
Anahtar kelimeler	`SELECT`, `select`, `Select` hepsi geçerli.
Takma ad (alias)	`Customer c` içindeki `c`. Zorunlu kabul et.
Sonuç	`SELECT c` → tüm entity nesnesi döner, kolon listesi değil.
Parametre	İsimli `:status` veya sıralı `?1`.

İlişkiler üzerinden gezinme — JPQL'in en büyük kolaylığı

JPQL

```
-- Gizli (implicit) join: noktayla gez
SELECT o FROM Order o
WHERE o.customer.email = :email

-- Açık join
SELECT o FROM Order o
JOIN o.customer c
WHERE c.status = 'ACTIVE'
```

SQL karşılığı

```
select o.* from orders o
join customers c
  on c.id = o.customer_id
where c.email = ?

-- ON koşulunu yazmana gerek kalmaz:
-- Hibernate @JoinColumn bilgisinden bilir.
```

JPQL kalıpları

```
-- Belirli alanlar (Object[] veya DTO döner)
SELECT c.id, c.email FROM Customer c

-- DTO'ya doğrudan eşleme (constructor expression): paket adıyla tam sınıf adı yazılır
SELECT new com.example.dto.CustomerDto(c.id, c.firstName, c.email) FROM Customer c

-- Toplama fonksiyonları ve gruplama
SELECT o.status, COUNT(o), SUM(o.total) FROM Order o GROUP BY o.status HAVING COUNT(o) > 10

-- Alt sorgu
SELECT c FROM Customer c WHERE NOT EXISTS (SELECT o FROM Order o WHERE o.customer = c)

-- Koleksiyon kontrolü
SELECT c FROM Customer c WHERE c.orders IS EMPTY
SELECT c FROM Customer c WHERE SIZE(c.orders) > 5

-- İlişkiyi tek sorguda birlikte yükleme (N+1 önleme)
SELECT o FROM Order o JOIN FETCH o.customer WHERE o.status = :status

-- CASE, COALESCE, fonksiyonlar
SELECT COALESCE(SUM(o.total), 0) FROM Order o WHERE o.customer.id = :id
SELECT c FROM Customer c WHERE LOWER(c.email) LIKE LOWER(:pattern)

-- Toplu güncelleme ve silme (Spring Data'da @Modifying gerekir)
UPDATE Customer c SET c.status = :to WHERE c.status = :from
DELETE FROM Order o WHERE o.createdAt < :before
```

JPQL'in sınırları

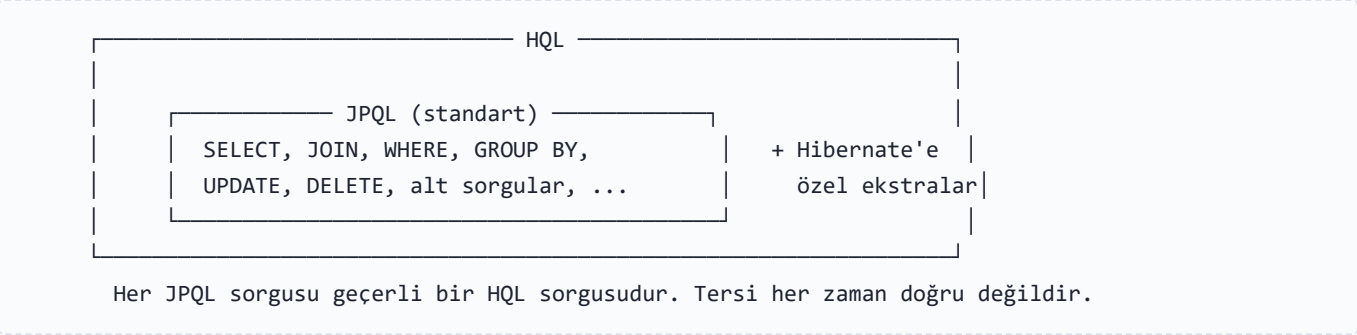
- Standart JPQL'de `LIMIT` yoktur. Sınırlamayı `setMaxResults()` veya Spring Data'da `Pageable` / `findTop10By…` ile yaparsın.
- Window fonksiyonları (`ROW_NUMBER() OVER …`), `WITH` (CTE), veritabanına özel operatörler (PostgreSQL'in JSON operatörleri gibi) standart JPQL'de yoktur. Bunların bir kısmını HQL destekler; desteklemediğinde native query'ye geçersin.
- Yalnızca entity'ler sorgulanabilir; entity olarak tanımlanmamış bir tabloyu JPQL ile sorgulayamazsın.

JPQL'in iki büyük avantajı

1) Veritabanı bağımsızlığı: Aynı sorgu PostgreSQL, MySQL ve Oracle'da çalışır. **2) Erken hata:** Spring Data'da `@Query` içindeki JPQL, uygulama açılırken doğrulanır. `c.fristName` gibi bir yazım hatası yaparsan uygulama hiç başlamaz; hatayı canlıda kullanıcı değil, sen görürsün.

2.7 HQL — Hibernate'in genişletilmiş sorgu dili

HQL (Hibernate Query Language), Hibernate'in kendi sorgu dilidir. Tarihsel sıra şöyledir: önce Hibernate HQL'i geliştirdi, sonra JPA standardı yazılırken HQL'in bir alt kümesi **JPQL** adıyla standartlaştırıldı.



Hibernate, JPQL ve HQL'i **aynı parser ile** işler. Spring Boot loglarında "HQL parser will be used" gibi bir satır görürsen sebebi budur: `@Query` içine yazdığın JPQL, Hibernate'in HQL parser'ından geçer.

HQL'in JPQL'e eklediği bazı şeyler (Hibernate 6+)

```
-- SELECT kısmı isteğe bağlı
from Customer where status = 'ACTIVE'

-- Sorgu içinde LIMIT / OFFSET
from Customer order by createdAt desc limit 10 offset 20

-- INSERT ... SELECT (JPQL'de INSERT yoktur)
insert into ArchivedOrder (id, total, createdAt)
select o.id, o.total, o.createdAt from Order o where o.createdAt < :before

-- Küme işlemleri
select c.email from Customer c union select s.email from Subscriber s

-- Daha zengin fonksiyonlar
select extract(year from o.createdAt), sum(o.total) from Order o group by 1
```

Ne zaman HQL'e özel bir şey yazarım?

Spring Data'nın `@Query` anotasyonu Hibernate ile çalıştığı için HQL'e özel söz dizimini de kabul eder. Ama günlük işlerin %95'inde standart JPQL yeterlidir. Taşınabilirlik önemliyse JPQL'de kal; gerçekten gerekirse HQL özelliklerini kullan. Pratikte "JPQL yazıyorum" dediğinde Hibernate kullanan bir projede aslında HQL yazıyorsundur; fark sadece ekstraları kullanıp kullanmamandır.

> **İsim karışıklığı: HQL ≠ HSQLDB**
> **HQL** bir sorgu dilidir. **HSQLDB (HyperSQL Database)** ise H2 gibi Java ile yazılmış, genelde testlerde kullanılan bellek içi bir veritabanıdır. İsimleri benzer ama aralarında hiçbir bağ yoktur.

2.8 Criteria API — sorguyu Java koduyla kurmak

Criteria API, JPA'nın sorguları string yerine **Java metot çağrılarıyla** kurmanı sağlayan parçasıdır. Asıl gücü, koşulların **çalışma anında** eklenip çıkarılabilmesidir (filtre ekranları). Spring Data'daki **Specification** bunun üzerine kuruludur.

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Customer> q = cb.createQuery(Customer.class);
Root<Customer> c = q.from(Customer.class);                  // FROM Customer c

List<Predicate> where = new ArrayList<>();
if (filter.status() != null) where.add(cb.equal(c.get("status"), filter.status()));
if (filter.name()   != null) where.add(cb.like(cb.lower(c.get("firstName")), "%" +
filter.name().toLowerCase() + "%"));

q.select(c).where(where.toArray(Predicate[]::new)).orderBy(cb.asc(c.get("firstName")));
List<Customer> result = em.createQuery(q).getResultList();
```

Sadece doldurulan filtreler sorguya eklenir. Aynı şeyi JPQL string'leriyle yapmaya çalışmak (`"... AND " + …`) hem çirkin hem hataya açıktır.

2.9 Native query — doğrudan SQL

Native query, JPA üzerinden veritabanına **düz SQL** göndermektir. Hibernate bu sorguyu çevirmez, olduğu gibi iletir. Tablo ve kolon adlarını kullanırsın.

Ne zaman gerekir?

- Window fonksiyonları, CTE (`WITH`), `LATERAL` gibi gelişmiş SQL özellikleri
- Veritabanına özel özellikler: PostgreSQL `ILIKE` , JSON/JSONB operatörleri, tam metin arama
- DBA'nın hazırladığı ve aynen kullanılması gereken optimize edilmiş bir sorgu
- Entity'si olmayan tablolar veya view'lar

Örnekler

```
// 1) EntityManager ile
List<Customer> list = em.createNativeQuery(
        "SELECT * FROM customers WHERE email ILIKE :p", Customer.class)
    .setParameter("p", "%@gmail.com")
    .getResultList();

// 2) Spring Data ile
@Query(value = "SELECT * FROM customers WHERE email ILIKE :p", nativeQuery = true)
List<Customer> searchEmail(@Param("p") String pattern);

// 3) Window fonksiyonu + arayüz projection: her müşterinin en büyük siparişi
public interface TopOrder { Long getCustomerId(); BigDecimal getTotal(); }

@Query(value = """
        SELECT customer_id AS customerId, total FROM (
            SELECT customer_id, total,
                   ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY total DESC) AS rn
            FROM orders
        ) t
        WHERE rn = 1
        """, nativeQuery = true)
List<TopOrder> findLargestOrderPerCustomer();
// Kolon takma adları (AS customerId) projection'daki getter adlarıyla eşleşmeli
```

Native query'nin bedeli

Risk	Açıklama
Veritabanı bağımlılığı	`ILIKE` MySQL'de çalışmaz. Veritabanı değişirse sorgular elden geçer.
Geç hata	Tablo/kolon adını yanlış yazarsan hatayı uygulama açılırken değil, sorgu ilk çalıştığında görürsün.
Refactor'a kör	Entity'de alan adını değiştirdiğinde IDE ve derleyici native SQL'deki kolon adını güncellemez.
Persistence context'i atlar	Native UPDATE/DELETE, hafızadaki entity'leri güncellemez (JPQL toplu sorgularıyla aynı durum).
Sayfalama	`Page` dönerken toplam sayı için `countQuery` 'yi genelde ayrıca yazman gerekir.

Kural

Önce JPQL ile dene. JPQL yetmiyorsa ve bu ihtiyaç gerçekse native'e geç. Native sorguları repository'de toplu tut ve testle koru.

2.10 Spring Data JPA — repository kolaylığı

Spring Data JPA, JPA'nın üstüne oturan ve tekrar eden veri erişim kodunu ortadan kaldıran Spring kütüphanesidir. Kendisi bir ORM değildir; arka planda JPA'yı (yani Hibernate'i) kullanır.

Aynı iş, iki yol

Saf JPA (EntityManager) — her şeyi sen yazarsın

```
@Repository
public class CustomerDao {
  @PersistenceContext EntityManager em;

  Customer save(Customer c) {
    if (c.getId() == null) {
      em.persist(c); return c;
    }
    return em.merge(c);
  }
  Optional<Customer> findById(Long id) {
    return Optional.ofNullable(
        em.find(Customer.class, id));
  }
  Optional<Customer> findByEmail(String e) {
    return em.createQuery(
      "SELECT c FROM Customer c WHERE c.email =
:e",
      Customer.class)
      .setParameter("e", e)
      .getResultStream().findFirst();
  }
  // findAll, delete, count, sayfalama...
}
```

Spring Data JPA — sadece arayüz

```
public interface CustomerRepository
    extends JpaRepository<Customer, Long> {

  Optional<Customer> findByEmail(String email);
}

// save, findById, findAll, delete,
// count, existsById, sayfalama, sıralama
// hepsi hazır.

// findByEmail'in sorgusunu
// metot adından Spring üretir.
```

Spring Data JPA neler sağlar?

Özellik	Örnek
Hazır CRUD	`save`, `findById`, `findAll`, `deleteById`, `count`
Metot adından sorgu	`findByStatusAndCreatedAtAfter(...)`
Elle sorgu	`@Query("SELECT ...")`, `nativeQuery = true`
Sayfalama ve sıralama	`Pageable`, `Sort`, `Page`, `Slice`
Projection	Arayüz veya record ile sadece gereken alanlar
Dinamik sorgu	`Specification`, Query by Example
Denetim (auditing)	`@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`

Perde arkası: arayüzün implementasyonu nereden geliyor?

1. Uygulama açılırken Spring, `JpaRepository`'den türeyen arayüzleri tarar.
2. Her biri için çalışma anında bir **proxy** nesnesi üretir ve bean olarak kaydeder. `@Autowired` ile aldığın şey bu proxy'dir.
3. Hazır metotlar (`save`, `findById` ...) `SimpleJpaRepository` sınıfına yönlendirilir; o da `EntityManager`'ı çağırır.

4. Senin tanımladığın `findByEmail` gibi metotların adı parçalanır ve JPQL'e çevrilir:

```
findByEmail(String email)
   ↓ Spring Data (PartTree)
select c from Customer c where c.email = ?1          -- JPQL
   ↓ Hibernate
select c.id, c.email, ... from customers c where c.email = ?   -- SQL
```

2.11 Aynı sorgu, yedi farklı yazım

Görev: Durumu `ACTIVE` olan ve e-postası `@gmail.com` ile biten müşterileri ada göre sıralı getir. Aşağıdaki yedi yazımın **hepsi** veritabanına neredeyse aynı SQL'i gönderir:

```
select c.id, c.created_at, c.email, c.first_name, c.status
from customers c
where c.status = ? and c.email like ?
order by c.first_name
```

1) JDBC

```
try (PreparedStatement ps = conn.prepareStatement(
        "SELECT * FROM customers WHERE status = ? AND email LIKE ? ORDER BY first_name")) {
    ps.setString(1, "ACTIVE");
    ps.setString(2, "%@gmail.com");
    ResultSet rs = ps.executeQuery();
    while (rs.next()) { list.add(mapRow(rs)); }   // eşleme elle
}
```

2) JPA — EntityManager + JPQL

```
List<Customer> list = em.createQuery(
        "SELECT c FROM Customer c WHERE c.status = :s AND c.email LIKE :d ORDER BY c.firstName",
        Customer.class)
    .setParameter("s", CustomerStatus.ACTIVE)
    .setParameter("d", "%@gmail.com")
    .getResultList();
```

3) Hibernate — Session + HQL

```
Session session = em.unwrap(Session.class);
List<Customer> list = session.createSelectionQuery(
        "from Customer where status = :s and email like :d order by firstName", Customer.class)
    .setParameter("s", CustomerStatus.ACTIVE)
    .setParameter("d", "%@gmail.com")
    .getResultList();
```

4) JPA — Criteria API

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Customer> q = cb.createQuery(Customer.class);
Root<Customer> c = q.from(Customer.class);
q.select(c)
 .where(cb.equal(c.get("status"), CustomerStatus.ACTIVE),
        cb.like(c.get("email"), "%@gmail.com"))
 .orderBy(cb.asc(c.get("firstName")));
List<Customer> list = em.createQuery(q).getResultList();
```

5) Spring Data JPA — metot adından sorgu

```
List<Customer> findByStatusAndEmailEndingWithOrderByFirstNameAsc(CustomerStatus status, String suffix);
// Çağrı: repo.findByStatusAndEmailEndingWithOrderByFirstNameAsc(ACTIVE, "@gmail.com")
// EndingWith, % işaretini kendisi ekler.
```

6) Spring Data JPA — @Query (JPQL)

```
@Query("SELECT c FROM Customer c WHERE c.status = :s AND c.email LIKE :d ORDER BY c.firstName")
List<Customer> findActiveByDomain(@Param("s") CustomerStatus s, @Param("d") String domainPattern);
```

7) Spring Data JPA — native query

```
@Query(value = "SELECT * FROM customers WHERE status = :s AND email LIKE :d ORDER BY first_name",
       nativeQuery = true)
List<Customer> findActiveByDomainNative(@Param("s") String s, @Param("d") String domainPattern);
// Native'de enum'u kendin string'e çeviririsin: "ACTIVE"
```

Karşılaştırma

Yöntem	Ne yazarsın?	Nesneye eşleme	DB bağımsız	Hata ne zaman?
JDBC	SQL	Elle	❌	Çalışırken
EntityManager + JPQL	JPQL	Otomatik	✅	Çalışırken
Session + HQL	HQL	Otomatik	✅	Çalışırken
Criteria API	Java kodu	Otomatik	✅	Derlemede (kısmen)
Spring Data metot adı	Metot adı	Otomatik	✅	**Açılışta**
Spring Data @Query	JPQL	Otomatik	✅	**Açılışta**
Spring Data native	SQL	Otomatik	❌	Çalışırken

2.12 Bir sorgunun yolculuğu: baştan sona

Service katmanında tek bir satır yazdığında arka planda neler olur?

```
Optional<Customer> c = customerRepository.findByEmail("ayse@example.com");
```

#	Katman	Ne olur?
0	Spring Data (açılışta)	Uygulama başlarken metot adı parçalanır, `select c from Customer c where c.email = ?1` JPQL'i üretilir ve doğrulanır. Alan adı yanlışsa uygulama burada açılmaz.
1	Spring proxy	`customerRepository` aslında Spring'in ürettiği bir proxy'dir. Çağrıyı yakalar ve sorgu yürütücüsüne iletir.
2	Transaction	Service'te açılmış bir transaction varsa ona katılır.
3	JPA / Hibernate (flush)	Hibernate, persistence context'te henüz DB'ye yazılmamış değişiklik varsa ve sorguyu etkileyebilecekse önce onları gönderir (auto flush). Böylece sorgu güncel veriyi görür.
4	Hibernate (çeviri)	JPQL, HQL parser'dan geçer ve PostgreSQL lehçesinde SQL'e çevrilir: `select c.id, c.email, ... from customers c where c.email = ?`
5	JDBC	HikariCP havuzundan bir `Connection` alınır, `PreparedStatement` hazırlanır, `"ayse@example.com"` parametre olarak bağlanır ve çalıştırılır.
6	Veritabanı	PostgreSQL sorguyu çalıştırır; `email` kolonundaki unique index sayesinde kaydı hızlıca bulur ve satırı döner.
7	Hibernate (hydration)	`ResultSet` 'teki satırdan bir `Customer` nesnesi oluşturulur. LAZY ilişkiler (`orders`) için yer tutucu proxy konur. Nesne persistence context'e eklenir, artık **managed**'dir.
8	Spring Data	Sonuç `Optional<Customer>` içine sarılıp service'e döner.
9	Sonrası	Service bu nesnenin bir alanını değiştirirse, transaction commit edilirken Hibernate farkı görür ve kendiliğinden `UPDATE` atar (dirty checking).

findById ve sorgu metodu arasındaki ince fark

`findById` önce persistence context'e bakar; nesne aynı transaction'da zaten yüklendiyse **veritabanına gitmez**. `findByEmail` gibi sorgu metotları ise her zaman SQL çalıştırır (sonuç zaten hafızadaki nesneyse aynı nesne döner).

2.13 Özet tablo

İsim	Ne?	Kim sağlar?	Örnek	Ne zaman kullanırsın?
SQL	Veritabanının dili	Veritabanı (PostgreSQL, MySQL...)	`SELECT * FROM customers WHERE email = ?`	Her zaman çalışan odur; doğrudan yazman native query veya JDBC'de gerekir
JDBC	Java ↔ DB bağlantı API'si	Java (`java.sql`) + DB sürücüsü	`PreparedStatement` , `ResultSet`	Doğrudan nadiren; tüm üst katmanlar onu kullanır
ORM	Nesne ↔ tablo eşleme fikri	Bir kavram	`persist(customer)` → INSERT	Nesnelerle çalışmak istediğinde
JPA	ORM şartnamesi / kurallar	Jakarta EE (`jakarta.persistence`)	`@Entity` , `@Column` , `EntityManager`	Entity tanımlarken, her zaman
Hibernate	JPA'yı uygulayan motor	Hibernate projesi (Red Hat)	DDL üretimi, JPQL → SQL, dirty checking	Arka planda hep çalışır; `@CreationTimestamp` gibi ekstralar için doğrudan
JPQL	JPA'nın sorgu dili (entity ve alan adlarıyla)	JPA	`SELECT c FROM Customer c WHERE c.status = :s`	`@Query` ile özel sorgu yazarken
HQL	Hibernate'in sorgu dili (JPQL'in üst kümesi)	Hibernate	`from Customer limit 10`	JPQL'in yetmediği, Hibernate'e özel durumlarda
Criteria API	Sorguyu Java koduyla kurmak	JPA	`cb.equal(root.get("status"), s)`	Dinamik filtrelerde (Specification ile)
Native query	`@Query` içinde düz SQL	JPA / Spring Data	`@Query(value = "...", nativeQuery = true)`	Veritabanına özel özellik gerektiğinde
Spring Data JPA	Repository kolaylığı	Spring	`JpaRepository` , `findByEmail`	Neredeyse her Spring projesinde, varsayılan seçim

Tek cümlelik özet

Sen **Spring Data JPA** ile bir arayüz yazarsın; o, **JPA** kurallarına göre **JPQL** üretir; **Hibernate** bunu (HQL parser'ı ile) veritabanının **SQL** lehçesine çevirir ve **JDBC** üzerinden veritabanına gönderir; dönen satırları da tekrar Java nesnesine dönüştürür.

3. Örnek entity'ler

Rehber boyunca bu iki sınıfı kullanacağız:

```
@Entity
@Table(name = "customers")
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false) private String firstName;
    @Column(nullable = false, unique = true) private String email;
    @Enumerated(EnumType.STRING) private CustomerStatus status;   // ACTIVE, PASSIVE, BANNED
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "customer")
    private List<Order> orders = new ArrayList<>();
}

@Entity
@Table(name = "orders")   // "order" SQL'de ayrılmış kelime
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "customer_id")
    private Customer customer;

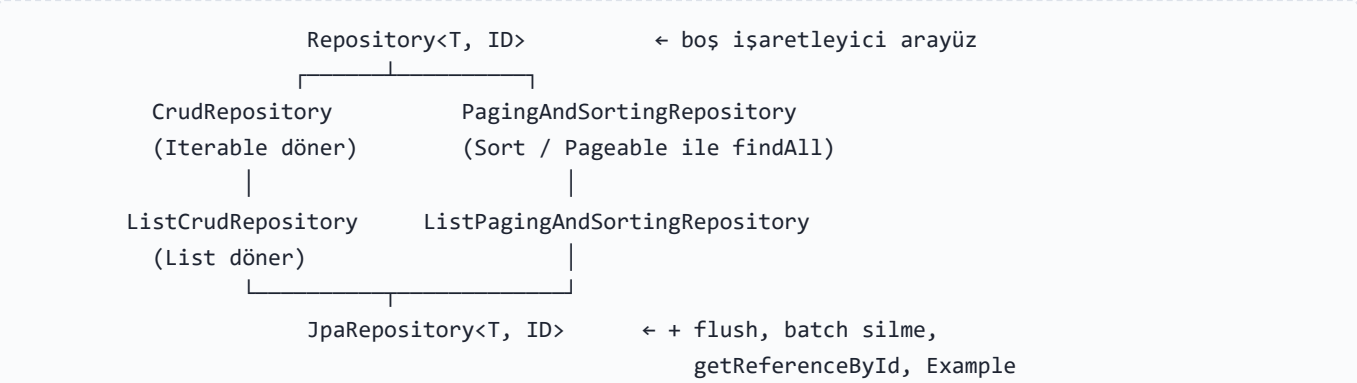
    private BigDecimal total;
    @Enumerated(EnumType.STRING) private OrderStatus status;     // NEW, PAID, CANCELLED
    private LocalDateTime createdAt;
}
```

4. Repository arayüz hiyerarşisi

Spring Data'da repository'yi **sınıf olarak değil, arayüz (interface) olarak** yazarsın. Uygulama açılırken Spring bu arayüzün implementasyonunu (bir proxy sınıfı) kendisi üretir ve bean olarak kaydeder.

```
public interface CustomerRepository extends JpaRepository<Customer, Long> { }
//                                                  │         └─ ID alanının tipi
//                                                  └─ Yönetilen entity
```

Bu tek satırla `save`, `findById`, `findAll`, `deleteById`, `count` ve daha fazlası hazır gelir. `@Repository` anotasyonu yazmak gerekmez; Spring arayüzü otomatik bulur.



Arayüz	Ne zaman?
`Repository`	Hiçbir hazır metot istemiyorsan; sadece kendi tanımladığın metotlar olsun.
`CrudRepository` / `ListCrudRepository`	Temel CRUD yeterliyse. List versiyonu `Iterable` yerine `List` döner.
`PagingAndSortingRepository`	Sayfalama ve sıralama gerekiyorsa (CRUD ile birlikte kullanılır).
`JpaRepository`	**En yaygın seçim.** Hepsini kapsar, JPA'ya özel ekstralar içerir.

Ek arayüzler

- `JpaSpecificationExecutor<T>`: dinamik sorgular (bölüm 13).
- `QueryByExampleExecutor<T>`: örnek bir nesneye göre arama (JpaRepository zaten içerir).

```
public interface OrderRepository
        extends JpaRepository<Order, Long>, JpaSpecificationExecutor<Order> { }
```

5. Hazır gelen metotlar

Metot	Ne yapar?
`save(entity)`	Yeni ise ekler (`persist`), değilse günceller (`merge`). Kaydedilmiş nesneyi döner.
`saveAll(list)`	Birden çok entity kaydeder.
`findById(id)`	`Optional<T>` döner. Kayıt yoksa boş Optional.
`existsById(id)`	Kayıt var mı? Tüm satırı yüklemez.
`findAll()` / `findAllById(ids)`	Tüm kayıtlar / verilen ID'ler. Büyük tablolarda `findAll()` tehlikelidir, sayfalama kullan.
`findAll(Sort)` / `findAll(Pageable)`	Sıralı / sayfalı listeleme.
`count()`	Toplam kayıt sayısı.
`deleteById(id)` / `delete(entity)`	Siler. Önce entity'yi yükler, sonra siler.
`deleteAllInBatch()` / `deleteAllByIdInBatch(ids)`	Tek bir DELETE sorgusuyla siler; entity yüklemez, cascade çalışmaz.
`getReferenceById(id)`	DB'ye gitmeden bir **proxy** döner. İlişki kurarken işe yarar (aşağıya bak).
`flush()` / `saveAndFlush(entity)`	Bekleyen değişiklikleri hemen DB'ye gönderir (transaction'ı bitirmez).

getReferenceById ne işe yarar?

```
// Sadece ilişki kurmak için müşterinin tüm alanlarını yüklemeye gerek yok:
Customer ref = customerRepository.getReferenceById(customerId);  // SELECT atılmaz
order.setCustomer(ref);
orderRepository.save(order);   // INSERT ... customer_id = ?
```

Dikkat

Proxy'nin bir alanına (`ref.getEmail()`) erişirsen o an SELECT atılır. Kayıt yoksa `EntityNotFoundException` alırsın. Kaydın varlığından emin değilsen `findById` kullan.

6. Metot adından sorgu (derived query)

Spring Data, metot adını okuyarak sorguyu kendisi üretir. Gövde yazmazsın.

```
findByStatusAndCreatedAtAfterOrderByCreatedAtDesc(status, date)
└─┘└┘└────┘└┘└──────┘└──────────────────────────┘
 │  │   │  │     │                └─ ORDER BY created_at DESC
 │  │   │  │     └─ created_at > ?
 │  │   │  └─ AND
 │  │   └─ status = ?
 │  └─ WHERE başlangıcı
 └─ SELECT
```

Başlangıç kelimeleri

Önek	Dönüş	Örnek
`find…By` , `get…By` , `read…By` , `query…By`	Entity, Optional, List…	`findByEmail(String email)`
`exists…By`	`boolean`	`existsByEmail(String email)`
`count…By`	`long`	`countByStatus(CustomerStatus s)`
`delete…By` , `remove…By`	`void` / `long`	`deleteByStatus(...)` (transaction gerekir)
`findFirst…By` , `findTop3…By`	İlk N kayıt	`findTop3ByOrderByTotalDesc()`
`findDistinct…By`	Tekrarsız	`findDistinctByStatus(...)`

Koşul anahtar kelimeleri

Anahtar kelime	Örnek metot	Üretilen koşul
`And` / `Or`	`findByFirstNameAndStatus`	`first_name = ? AND status = ?`
`Not`	`findByStatusNot`	`status <> ?`
`LessThan`, `LessThanEqual`	`findByTotalLessThan`	`total < ?`
`GreaterThan`, `GreaterThanEqual`	`findByTotalGreaterThanEqual`	`total >= ?`
`Between`	`findByCreatedAtBetween`	`created_at BETWEEN ? AND ?`
`Before` / `After`	`findByCreatedAtBefore`	`created_at < ?`
`IsNull` / `IsNotNull`	`findByDeletedAtIsNull`	`deleted_at IS NULL`
`Like`	`findByEmailLike("%@gmail.com")`	`email LIKE ?` (% işaretini sen verirsin)
`StartingWith` / `EndingWith` / `Containing`	`findByFirstNameContaining("ay")`	`first_name LIKE '%ay%'`
`IgnoreCase`	`findByEmailIgnoreCase`	`UPPER(email) = UPPER(?)`
`In` / `NotIn`	`findByStatusIn(List<…>)`	`status IN (?, ?)`
`True` / `False`	`findByActiveTrue`	`active = true`
`OrderBy…Asc/Desc`	`findByStatusOrderByCreatedAtDesc`	`ORDER BY created_at DESC`

İlişkiler üzerinden gezinme

```
// Order → customer → id   (alt çizgi ayrımı netleştirir, isteğe bağlı)
List<Order> findByCustomerId(Long customerId);
List<Order> findByCustomer_Email(String email);   // JOIN customers ... WHERE c.email = ?

// Sahiplik kontrolü için çok kullanışlı: başkasının siparişi bulunamaz
Optional<Order> findByIdAndCustomerId(Long id, Long customerId);
```

Ne zaman derived query?
Kısa, okunur koşullar: 1–3 alan, basit karşılaştırmalar. Hızlı yazılır, alan adı yanlışsa uygulama açılırken hata verir.

Ne zaman bırakmalı?
Metot adı `findByAAndBOrCAndDNotInOrderBy…` gibi uzadıysa okunmaz hale gelmiştir. `@Query` veya Specification'a geç.

7. @Query ile JPQL

Sorguyu kendin yazmak istediğinde `@Query` kullanırsın. Varsayılan dil **JPQL**'dir: tablo yerine entity, kolon yerine Java alan adı.

```
@Query("SELECT c FROM Customer c WHERE c.status = :status AND c.createdAt >= :since")
List<Customer> findRecentByStatus(@Param("status") CustomerStatus status,
                                  @Param("since") LocalDateTime since);
```

JPQL	SQL karşılığı
`Customer c` (entity adı)	`customers` (tablo adı)
`c.createdAt` (alan adı)	`created_at` (kolon adı)
`o.customer.id` (ilişki üzerinden)	`customer_id`
`:status` (isimli parametre)	`?`

Parametre bağlama

- **İsimli:** `:status` ve `@Param("status")`. Okunaklı, önerilen yol.
- **Sıralı:** `?1`, `?2`. Parametre sırasına bağlıdır, kırılgandır.
- Parametreler sorguya metin olarak eklenmez, ayrı gönderilir. Bu yüzden **SQL injection'a karşı güvenlidir**. Sorguyu asla string birleştirerek kurma.

Faydalı JPQL kalıpları

```
// JOIN
@Query("SELECT o FROM Order o JOIN o.customer c WHERE c.email = :email")

// Toplama fonksiyonları
@Query("SELECT COALESCE(SUM(o.total), 0) FROM Order o WHERE o.customer.id = :id")
BigDecimal totalSpent(@Param("id") Long customerId);

// GROUP BY ve DTO'ya doğrudan eşleme (constructor expression)
@Query("SELECT new com.example.dto.StatusCount(o.status, COUNT(o)) " +
       "FROM Order o GROUP BY o.status")
List<StatusCount> countByStatus();

// Koleksiyon parametresi
@Query("SELECT c FROM Customer c WHERE c.status IN :statuses")
List<Customer> findByStatuses(@Param("statuses") Collection<CustomerStatus> statuses);
```

Avantajı
JPQL veritabanından bağımsızdır; PostgreSQL'den MySQL'e geçsen sorgu aynı kalır. Ayrıca entity veya alan adını yanlış yazarsan hata **uygulama açılırken** çıkar, canlıda sürpriz yaşamazsın.

8. Native query

Veritabanına özel bir özellik (PostgreSQL'in JSON operatörleri, window fonksiyonları vb.) gerektiğinde düz SQL yazarsın:

```
@Query(value = "SELECT * FROM customers WHERE email ILIKE :pattern", nativeQuery = true)
List<Customer> searchByEmail(@Param("pattern") String pattern);

// Sayfalama yaparken toplam sayıyı bulacak sorguyu da vermen gerekir
@Query(value = "SELECT * FROM orders WHERE status = :status",
       countQuery = "SELECT COUNT(*) FROM orders WHERE status = :status",
       nativeQuery = true)
Page<Order> findByStatusNative(@Param("status") String status, Pageable pageable);
```

	JPQL	Native SQL
İsimler	Entity / alan	Tablo / kolon
DB bağımsız	✅	❌
Hata ne zaman çıkar	Uygulama açılırken	Sorgu çalıştığında
DB'ye özel özellikler	Sınırlı	✅ Tamamı

Kural: Önce JPQL dene; gerçekten yetmiyorsa native'e geç.

9. Veri değiştiren sorgular: @Modifying

Çok sayıda satırı tek seferde güncellemek veya silmek için `UPDATE` / `DELETE` sorgusu yazarsın:

```
@Modifying
@Query("UPDATE Customer c SET c.status = :newStatus WHERE c.status = :oldStatus")
int changeStatus(@Param("oldStatus") CustomerStatus oldStatus,
                 @Param("newStatus") CustomerStatus newStatus);   // etkilenen satır sayısı

@Modifying
@Query("DELETE FROM Order o WHERE o.status = :status AND o.createdAt < :before")
int purge(@Param("status") OrderStatus status, @Param("before") LocalDateTime before);
```

Bilmen gerekenler

- **`@Modifying` zorunlu.** Olmazsa Spring sorgudan sonuç bekler ve "must be a select query" hatası verir.
- **Transaction zorunlu.** Çağıran service metodu `@Transactional` olmalı, yoksa `TransactionRequiredException` alırsın.
- **Persistence context'i atlar.** Sorgu doğrudan DB'ye gider. Aynı transaction'da önceden yüklenmiş nesneler eski değerleri gösterir. Çözüm: `@Modifying(clearAutomatically = true)`.
- **Bekleyen değişiklikler:** Önce hafızadaki değişikliklerin DB'ye yazılmasını istiyorsan `flushAutomatically = true`.
- **Entity olayları çalışmaz.** `@PreUpdate`, `@UpdateTimestamp`, `@Version` artışı, cascade gibi entity seviyesindeki mekanizmalar atlanır.

Toplu güncelleme mi, tek tek mi?

	Yükle → değiştir → kaydet	@Modifying UPDATE
Sorgu sayısı	1 SELECT + N UPDATE	1 UPDATE
Bellek	Tüm satırlar Java'ya yüklenir	Hiçbiri yüklenmez
Entity olayları / @Version	✅ Çalışır	❌ Atlanır
Uygun olduğu yer	Az kayıt, iş kuralı gereken durumlar	Çok kayıt, basit toplu değişiklik

Derived delete tuzağı

`deleteByStatus(...)` gibi metot adından türetilen silmeler önce tüm kayıtları **yükler**, sonra **tek tek** siler. Binlerce kayıt için yavaştır. Büyük silmeler için `@Modifying @Query("DELETE ...")` kullan.

10. Dönüş tipleri

Tip	Ne zaman?
`Optional<T>`	Tek kayıt ve **bulunamayabilir**. Önerilen yol. `null` kontrolünü unutmanı engeller.
`T`	Tek kayıt; bulunamazsa `null` döner. Optional tercih edilmeli.
`List<T>`	Birden çok kayıt. Sonuç yoksa **boş liste** (asla `null` değil).
`Page<T>`	Sayfalı sonuç ve toplam kayıt sayısı (ekstra COUNT sorgusu atar).
`Slice<T>`	Sayfalı sonuç, sadece "sonraki sayfa var mı?" bilgisi. COUNT atmaz, daha ucuz. Sonsuz kaydırma için ideal.
`Stream<T>`	Çok büyük sonuçları parça parça işlemek. Transaction içinde kullanılmalı ve `try-with-resources` ile kapatılmalı.
`boolean` / `long`	`existsBy…` / `countBy…` .
Projection / DTO	Sadece bazı alanlar gerektiğinde (bölüm 12).

Hata

Tek kayıt dönen bir metot (`Optional<T>` veya `T`) birden fazla satır bulursa `IncorrectResultSizeDataAccessException` fırlatır. Benzersiz olmayan alanlarla tek kayıt arama.

11. Sayfalama ve sıralama

Binlerce kaydı tek seferde döndürmek hem DB'yi hem ağı yorar. Sayfalama ile parça parça getirirsin.

```
// Repository
Page<Order> findByCustomerId(Long customerId, Pageable pageable);

// Service: 0. sayfa (ilk sayfa), sayfa başına 20 kayıt, en yeniden eskiye
Pageable pageable = PageRequest.of(0, 20, Sort.by("createdAt").descending());
Page<Order> page = orderRepository.findByCustomerId(customerId, pageable);

page.getContent();        // bu sayfadaki kayıtlar
page.getTotalElements();  // toplam kayıt sayısı
page.getTotalPages();     // toplam sayfa
page.hasNext();           // sonraki sayfa var mı
```

Sadece sıralama

```
List<Customer> findByStatus(CustomerStatus status, Sort sort);
// Çağrı:
repo.findByStatus(ACTIVE, Sort.by("firstName").ascending().and(Sort.by("createdAt").descending()));
```

Üretilen SQL

```
SELECT ... FROM orders WHERE customer_id = ? ORDER BY created_at DESC LIMIT 20 OFFSET 0;
SELECT COUNT(*) FROM orders WHERE customer_id = ?;   -- Page için ek sorgu
```

> **Dikkat**
> Sayfa numarası **0'dan başlar**. Kullanıcıdan gelen sıralama alanını doğrudan `Sort.by(...)` 'a verme; izin verilen alanlar listesiyle kontrol et. Aksi halde var olmayan bir alanla hata alırsın ya da beklenmedik kolonlara göre sıralama yapılabilir.

12. Projection: sadece gereken alanlar

Bir listede sadece ad ve e-posta gösteriyorsan tüm entity'yi yüklemek gereksizdir. Projection ile sadece istediğin kolonları çekersin.

Arayüz tabanlı projection

```
public interface CustomerSummary {
    Long getId();
    String getFirstName();
    String getEmail();
}

List<CustomerSummary> findByStatus(CustomerStatus status);
// SELECT id, first_name, email FROM customers WHERE status = ?
```

DTO / record tabanlı projection

```
public record CustomerDto(Long id, String firstName, String email) { }

List<CustomerDto> findAllByStatus(CustomerStatus status);   // record alan adları entity ile eşleşmeli

// veya JPQL constructor expression ile:
@Query("SELECT new com.example.dto.CustomerDto(c.id, c.firstName, c.email) FROM Customer c")
List<CustomerDto> findAllSummaries();
```

Dinamik projection

```
<T> List<T> findByStatus(CustomerStatus status, Class<T> type);
// repo.findByStatus(ACTIVE, CustomerSummary.class)
```

Bonus

Projection'lar **yönetilen entity değildir**; değiştirip kaydedemezsin. Bu, salt okunur ekranlar için tam olarak istenen şeydir ve persistence context'i şişirmez.

13. Dinamik sorgular: Specification

Filtre ekranlarında hangi koşulların geleceği belli değildir: kullanıcı bazen sadece durumu, bazen tarih aralığını ve tutarı seçer. Her kombinasyon için ayrı metot yazmak imkânsızdır. **Specification** koşulları parça parça birleştirmeni sağlar.

```
public interface OrderRepository
        extends JpaRepository<Order, Long>, JpaSpecificationExecutor<Order> { }

public final class OrderSpecs {
    public static Specification<Order> hasStatus(OrderStatus status) {
        return (root, query, cb) -> status == null ? null : cb.equal(root.get("status"), status);
    }
    public static Specification<Order> totalAtLeast(BigDecimal min) {
        return (root, query, cb) -> min == null ? null : cb.greaterThanOrEqualTo(root.get("total"),
min);
    }
    public static Specification<Order> belongsTo(Long customerId) {
        return (root, query, cb) -> cb.equal(root.get("customer").get("id"), customerId);
    }
}

// Service: null dönen koşullar otomatik atlanır
Specification<Order> spec = Specification.allOf(
        OrderSpecs.belongsTo(customerId),
        OrderSpecs.hasStatus(filter.status()),
        OrderSpecs.totalAtLeast(filter.minTotal()));
Page<Order> result = orderRepository.findAll(spec, pageable);
```

- `root` : sorgulanan entity (`FROM Order o` 'daki `o`).
- `cb` (CriteriaBuilder): koşul üretir: `equal` , `like` , `between` , `and` , `or` ...
- Bir Specification `null` dönerse "koşul yok" anlamına gelir; filtre boşsa bu sayede atlanır.
- Alan adlarını string yerine tip güvenli yazmak istersen JPA Static Metamodel (`Order_.status`) kullanılabilir.

Alternatifler

Yöntem	Ne zaman?
Derived query	Koşullar sabit ve az
@Query	Koşullar sabit ama sorgu karmaşık
Specification	Koşullar çalışma anında değişiyor (filtre ekranları)
Query by Example	Basit eşitlik aramaları: "bu nesnedeki dolu alanlara uyanları getir"
Querydsl / jOOQ	Çok karmaşık, tip güvenli dinamik sorgular (harici kütüphane)

14. Transaction yönetimi

Transaction, birden çok veritabanı işleminin "ya hep ya hiç" çalışmasıdır. Sipariş oluşturup stoğu düşürürken biri başarısız olursa diğeri de geri alınmalıdır.

```
@Service
public class OrderService {

    @Transactional                          // metot başında başlar, sonunda commit edilir
    public Order placeOrder(Long customerId, BigDecimal total) {
        Customer c = customerRepository.findById(customerId).orElseThrow();
        Order order = new Order(c, total);
        orderRepository.save(order);
        c.setLastOrderAt(LocalDateTime.now());   // save çağırmaya gerek yok (dirty checking)
        return order;
    }                                           // RuntimeException fırlarsa hepsi geri alınır

    @Transactional(readOnly = true)          // sadece okuma: daha hafif çalışır
    public List<Order> history(Long customerId) { ... }
}
```

Kurallar

- `@Transactional` **service katmanına konur**, repository'ye değil. Bir iş işlemi genelde birden çok repository çağrısı içerir ve hepsi aynı transaction'da olmalıdır.
- Spring Data'nın hazır metotları (`save` , `findById` ...) zaten kendi transaction'ı ile gelir. Ama birden çok çağrıyı birleştirmek için service'te `@Transactional` gerekir.
- Varsayılan olarak **sadece unchecked exception'lar** (`RuntimeException`) geri alma (rollback) yapar. Checked exception'lar için `rollbackFor = Exception.class` .
- `readOnly = true` okuma metotlarında kullanılmalı: Hibernate değişiklik takibini atlar, bazı veritabanları optimizasyon yapar.

Klasik tuzak: self-invocation

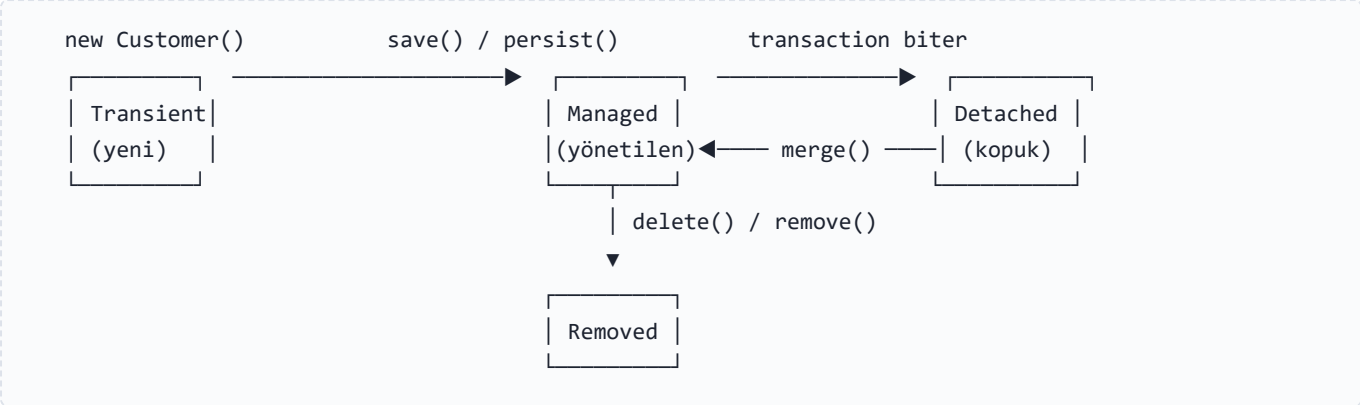
`@Transactional` bir proxy üzerinden çalışır. Aynı sınıf içinden `this.otherMethod()` çağırırsan proxy atlanır ve `otherMethod` üzerindeki `@Transactional` **çalışmaz**. Ayrıca `private` metotlarda da çalışmaz. Transaction gereken metot `public` olmalı ve dışarıdan çağrılmalı.

Propagation (kısaca)

Değer	Davranış
`REQUIRED` (varsayılan)	Varsa mevcut transaction'a katılır, yoksa yenisini açar.
`REQUIRES_NEW`	Her zaman yeni ve bağımsız transaction açar. Ana işlem geri alınsa bile kalması gereken log/denetim kayıtları için.
`MANDATORY`	Mevcut bir transaction yoksa hata verir.

15. Persistence context ve entity yaşam döngüsü

Hibernate, bir transaction boyunca yüklediği entity'leri **persistence context** (birinci seviye önbellek) adı verilen bir hafızada tutar. Repository'nin "sihirli" görünen davranışlarının çoğu buradan gelir.



Durum	Anlamı
Transient	`new` ile oluşturuldu, Hibernate'in haberi yok, DB'de karşılığı yok.
Managed	Persistence context içinde. Üzerindeki her değişiklik takip edilir.
Detached	DB'de karşılığı var ama transaction bitti; değişiklikler artık takip edilmiyor.
Removed	Silinmek üzere işaretlendi; commit'te DELETE atılır.

Dirty checking: save çağırmadan güncelleme

```
@Transactional
public void rename(Long id, String newName) {
    Customer c = customerRepository.findById(id).orElseThrow();  // managed
    c.setFirstName(newName);
    // save() yok! Commit anında Hibernate değişikliği fark eder ve UPDATE atar.
}
```

Birinci seviye önbellek

```
@Transactional
public void demo() {
    customerRepository.findById(1L);   // SELECT atılır
    customerRepository.findById(1L);   // SELECT atılmaz, hafızadan gelir
}
```

save() aslında ne yapar?

- Entity **yeni** ise (ID'si `null`): `persist` → INSERT. Aynı nesne managed olur.
- Entity yeni **değilse**: `merge` → DB'deki kaydı yükler, değerleri kopyalar ve **yeni bir managed kopya döner**. Bu yüzden `entity = repo.save(entity)` şeklinde dönüş değerini kullanmak güvenlidir.

LazyInitializationException

LAZY bir ilişkiye (`order.getCustomer().getEmail()`) transaction bittikten sonra, yani entity detached iken erişirsen bu hatayı alırsın. Çözüm: veriyi transaction içinde hazırla (fetch join, entity graph veya DTO). `spring.jpa.open-in-view` ayarını açık bırakmak sorunu gizler ama sorgu sayısını kontrol edilemez hale getirir; kapatman önerilir.

16. Performans: N+1, LAZY/EAGER, fetch join

LAZY ve EAGER

	LAZY	EAGER
Ne zaman yüklenir?	İlk erişildiğinde	Ana entity ile birlikte, hemen
Varsayılan	`@OneToMany`, `@ManyToMany`	`@ManyToOne`, `@OneToOne`
Öneri	✅ **Her ilişkide LAZY kullan**	❌ Gereksiz sorgular, kontrol kaybı

`@ManyToOne` varsayılan olarak EAGER'dır. Bu yüzden `fetch = FetchType.LAZY` 'yi açıkça yazmak iyi bir alışkanlıktır. İhtiyaç olduğunda ilişkiyi sorgu bazında yüklersin.

N+1 problemi

```
List<Order> orders = orderRepository.findAll();          // 1 sorgu: 100 siparis
for (Order o : orders) {
    System.out.println(o.getCustomer().getEmail());       // her biri için 1 sorgu: +100
}
// Toplam 101 sorgu! Liste büyüdükçe uygulama yavaşlar.
```

Çözüm 1: JOIN FETCH

```
@Query("SELECT o FROM Order o JOIN FETCH o.customer WHERE o.status = :status")
List<Order> findWithCustomerByStatus(@Param("status") OrderStatus status);
// Tek sorgu: siparişler ve müşterileri birlikte gelir
```

Çözüm 2: @EntityGraph

```
@EntityGraph(attributePaths = "customer")
List<Order> findByStatus(OrderStatus status);   // derived query'yi bozmadan ilişkiyi yükler
```

Çözüm 3: Projection / DTO

Sadece birkaç alan gerekiyorsa en verimli yol, entity yerine DTO çekmektir (bölüm 12).

> **Koleksiyon fetch join'i ve sayfalama**
> `JOIN FETCH c.orders` gibi bir **koleksiyonu** fetch join ile çekip aynı anda sayfalama yaparsan Hibernate sayfalamayı bellekte yapar (uyarı loglar) ve tüm satırları yükler. Bu durumda önce ID'leri sayfalı çek, sonra ikinci sorguda ilişkileri yükle; veya `@BatchSize` / `hibernate.default_batch_fetch_size` kullan.

Diğer performans ipuçları

- **Index:** WHERE, JOIN ve ORDER BY'da sık kullanılan kolonlara index ekle (ör. `customer_id`, `status`). `unique` kısıtı otomatik index oluşturur.
- **exists / count:** Sadece varlık kontrolü için kayıt yükleme, `existsBy…` kullan.

- **Sorguları gör:** Geliştirmede `spring.jpa.show-sql=true` ile kaç sorgu atıldığını izle. N+1 genelde loglarda tekrarlanan aynı SELECT olarak görünür.
- **Batch insert:** Çok sayıda kayıt eklerken `hibernate.jdbc.batch_size` ayarı sorguları gruplar. `IDENTITY` ID stratejisi batch insert'i engeller; yoğun yazma gereken tablolarda `SEQUENCE` tercih edilir.

17. Kilitleme: optimistic ve pessimistic

İki kullanıcı aynı kaydı aynı anda değiştirirse, birinin değişikliği diğerini sessizce ezebilir (**lost update**).

Optimistic locking: @Version

```
@Entity
public class Product {
    @Id Long id;
    Integer stock;
    @Version Long version;    // her güncellemede Hibernate 1 artırır
}
// UPDATE products SET stock = ?, version = 6 WHERE id = ? AND version = 5
// Başkası arada güncelledi̇yse 0 satır etkilenir → OptimisticLockException
```

Çakışma **nadirse** idealdir: kilit tutulmaz, çakışma olursa hata alınır ve işlem tekrar denenir veya kullanıcıya bildirilir.

Pessimistic locking: @Lock

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT p FROM Product p WHERE p.id = :id")
Optional<Product> findByIdForUpdate(@Param("id") Long id);
// SELECT ... FOR UPDATE → transaction bitene kadar başkası bu satırı değiştiremez
```

Çakışma **sıksa** ve kesinlikle önlenmesi gerekiyorsa (stok düşme, bakiye) kullanılır. Bedeli: diğer işlemler bekler, yanlış kullanımda deadlock riski vardır. Transaction içinde çağrılmalıdır.

18. Özel repository implementasyonu

Derived query, @Query ve Specification yetmediğinde (ör. `EntityManager` veya JDBC ile özel bir iş) repository'ye kendi kodunu ekleyebilirsin:

```
// 1) Ek metotları tanımlayan arayüz
public interface CustomerRepositoryCustom {
    List<Customer> searchComplex(SearchCriteria criteria);
}

// 2) Implementasyon: ad mutlaka arayüz adı + "Impl" olmalı
class CustomerRepositoryCustomImpl implements CustomerRepositoryCustom {
    @PersistenceContext private EntityManager em;

    @Override
    public List<Customer> searchComplex(SearchCriteria criteria) {
        // em.createQuery(...) ile istediğin sorguyu kur
    }
}

// 3) Ana repository ikisini birleştirir
public interface CustomerRepository
        extends JpaRepository<Customer, Long>, CustomerRepositoryCustom { }
```

Service tarafında tek bir `CustomerRepository` görürsün; hazır metotlar da özel metotlar da aynı yerden çağrılır.

19. Repository testleri

`@DataJpaTest` sadece JPA katmanını (entity'ler, repository'ler, veritabanı bağlantısı) ayağa kaldırır. Controller ve service yüklenmez; testler hızlıdır ve her test sonunda transaction geri alınır.

```
@DataJpaTest
class OrderRepositoryTest {

    @Autowired OrderRepository orderRepository;
    @Autowired TestEntityManager em;

    @Test
    void findByIdAndCustomerId_returnsEmpty_forOtherCustomersOrder() {
        Customer owner = em.persist(new Customer("Ayşe", "ayse@example.com"));
        Customer other = em.persist(new Customer("Ali", "ali@example.com"));
        Order order = em.persist(new Order(owner, new BigDecimal("100")));
        em.flush();
        em.clear();   // önbelleği temizle, sorgu gerçekten DB'ye gitsin

        assertThat(orderRepository.findByIdAndCustomerId(order.getId(), other.getId())).isEmpty();
        assertThat(orderRepository.findByIdAndCustomerId(order.getId(), owner.getId())).isPresent();
    }
}
```

Hangi veritabanıyla test?

Seçenek	Artı	Eksi
H2 (bellek içi)	Kurulum yok, çok hızlı	Gerçek DB'den farklı davranabilir; native sorgular kırılabilir
Testcontainers	Gerçek PostgreSQL/MySQL'i Docker'da çalıştırır, canlıyla aynı davranış	Docker gerekir, biraz daha yavaş

Neyi test etmeli?

- Kendi yazdığın sorgular: `@Query`, karmaşık derived query'ler, Specification'lar, `@Modifying` sorgular.
- Sahiplik ve güvenlik filtreleri (başkasının kaydı dönmüyor mu?).
- `save`, `findById` gibi hazır metotları test etmeye gerek yok; onları Spring zaten test ediyor.

20. Sık yapılan hatalar

Hata	Sonuç	Doğrusu
Repository'de iş kuralı yazmak	Katmanlar karışır, test zorlaşır	İş kuralları service'te
Metot adında alanı yanlış yazmak (`findByEmial`)	Uygulama açılmaz: `No property 'emial' found`	Alan adlarıyla birebir eşleştir
JPQL'de tablo/kolon adı kullanmak	Sorgu derlenmez	Entity ve alan adları kullan
`@Modifying`'i unutmak	"must be a select query" hatası	UPDATE/DELETE'te `@Modifying`
Değiştiren sorguyu transaction dışında çağırmak	`TransactionRequiredException`	Service metodunda `@Transactional`
Optional'ı `.get()` ile açmak	Kayıt yoksa `NoSuchElementException`	`.orElseThrow(() -> new NotFoundException(...))`
Büyük tabloda `findAll()`	Bellek taşar, yavaşlık	Sayfalama veya filtre
Her ilişkide EAGER	Gereksiz JOIN ve sorgular	LAZY + gerektiğinde fetch join
Döngü içinde LAZY ilişkiye erişmek	N+1 sorgu	JOIN FETCH / EntityGraph / DTO
Entity'yi doğrudan API'den döndürmek	LAZY hataları, sonsuz JSON döngüsü, gizli alanların sızması	DTO'ya çevir
Sorguyu string birleştirerek kurmak	SQL injection	Parametre bağlama (`:param`)
Sahiplik kontrolünü unutmak (`findById`)	Kullanıcı başkasının verisini görür	`findByIdAndOwnerId`
Aynı sınıftan `@Transactional` metodu çağırmak	Transaction açılmaz	Ayrı bean'e taşı veya dışarıdan çağır

21. Kopya kâğıdı

Hangi sorgu yöntemini seçmeliyim?

```
Koşullar çalışma anında mı değişiyor? (filtre ekranı)
   ├─ Evet → Specification
   └─ Hayır → Metot adı kısa ve okunur kalıyor mu?
               ├─ Evet → Derived query (findBy…)
               └─ Hayır → JPQL yeterli mi?
                           ├─ Evet → @Query (JPQL)
                           └─ Hayır → @Query(nativeQuery = true)

Çok satırı tek seferde mi değiştiriyorum? → @Modifying @Query("UPDATE/DELETE …")
Sadece birkaç alan mı lazım?            → Projection / DTO
İlişkiyi de birlikte mi lazım?          → JOIN FETCH / @EntityGraph
```

Tipik bir repository

```java
public interface OrderRepository
        extends JpaRepository<Order, Long>, JpaSpecificationExecutor<Order> {

    // Sahiplik kontrolü
    Optional<Order> findByIdAndCustomerId(Long id, Long customerId);
    boolean existsByIdAndCustomerId(Long id, Long customerId);

    // Sayfalı listeleme
    Page<Order> findByCustomerId(Long customerId, Pageable pageable);

    // İlişkiyle birlikte (N+1 önleme)
    @EntityGraph(attributePaths = "customer")
    List<Order> findByStatus(OrderStatus status);

    // Toplama
    @Query("SELECT COALESCE(SUM(o.total), 0) FROM Order o WHERE o.customer.id = :customerId")
    BigDecimal totalSpent(@Param("customerId") Long customerId);

    // Toplu güncelleme
    @Modifying(clearAutomatically = true)
    @Query("UPDATE Order o SET o.status = :to WHERE o.status = :from AND o.createdAt < :before")
    int bulkChangeStatus(@Param("from") OrderStatus from, @Param("to") OrderStatus to,
                         @Param("before") LocalDateTime before);
}
```

Akılda kalması gereken 10 madde

1. Repository sadece veri erişimi yapar; iş kuralı service'tedir.
2. Arayüz yaz, implementasyonu Spring üretir. Çoğu zaman `JpaRepository` yeterlidir.
3. Tek kayıt aramalarında `Optional` dön.
4. JPQL'de entity ve alan adları kullanılır, SQL'de tablo ve kolon adları.
5. UPDATE / DELETE sorgularında `@Modifying` ve transaction gerekir.

6. `@Transactional` service katmanına konur; okumalarda `readOnly = true`.
7. Managed entity'yi değiştirmek yeterlidir; dirty checking UPDATE'i kendisi atar.
8. İlişkileri LAZY yap, gerektiğinde fetch join veya entity graph ile yükle.
9. Büyük listeleri sayfala; sadece gereken alanlar için projection kullan.
10. Geliştirirken `show-sql` ile atılan sorguları izle; N+1'i erken yakala.

22. Terimler sözlüğü

Terim	Anlamı
Entity	`@Entity` ile işaretlenmiş, bir tabloya karşılık gelen Java sınıfı.
Repository	Bir entity için veri erişim işlemlerini toplayan katman / arayüz.
EntityManager	JPA'nın ana arayüzü: `persist`, `find`, `merge`, `remove`, sorgu oluşturma.
Session	Hibernate'in EntityManager karşılığı. `em.unwrap(Session.class)` ile erişilir.
Persistence context	Bir transaction boyunca yüklenen entity'lerin tutulduğu hafıza (birinci seviye önbellek).
Managed / Detached	Persistence context tarafından takip edilen / artık takip edilmeyen entity.
Dirty checking	Managed entity'deki değişikliklerin otomatik algılanıp commit'te UPDATE'e çevrilmesi.
Flush	Hafızadaki bekleyen değişiklikleri SQL olarak veritabanına göndermek. Transaction'ı bitirmez.
Commit / Rollback	Transaction'daki değişiklikleri kalıcı yapmak / tamamen geri almak.
Transaction	"Ya hep ya hiç" çalışan işlemler bütünü.
Proxy	Gerçek nesnenin yerine geçen ara nesne. Spring repository'leri ve Hibernate LAZY ilişkileri proxy kullanır.
Lazy / Eager loading	İlişkili verinin ilk erişimde / ana entity ile birlikte hemen yüklenmesi.
N+1 problemi	1 liste sorgusu ve her eleman için ayrı birer sorgu atılması.
Fetch join	İlişkiyi aynı sorguda birlikte yükleyen JOIN (`JOIN FETCH`).
Dialect (lehçe)	Hibernate'in hangi veritabanının SQL çeşidini üreteceğini belirleyen ayar.
DDL / DML	Yapıyı tanımlayan (CREATE, ALTER, DROP) / veriyi işleyen (SELECT, INSERT, UPDATE, DELETE) SQL komutları.
ddl-auto	Hibernate'in şemayı otomatik oluşturma/güncelleme ayarı.
Migration (Flyway / Liquibase)	Şema değişikliklerini sürümlü SQL dosyalarıyla yönetme yöntemi. Canlı ortamda `ddl-auto` yerine kullanılır.
Connection pool (HikariCP)	Hazır veritabanı bağlantılarını tekrar kullanan havuz.
PreparedStatement	Parametreleri ayrı gönderilen, önceden derlenmiş SQL. SQL injection'a karşı korur.
ResultSet	JDBC sorgu sonucunun satır satır okunduğu nesne.
Hydration	Veritabanı satırlarından entity nesnesi oluşturma işlemi.
Derived query	Metot adından Spring Data'nın ürettiği sorgu (`findByEmail`).
Projection	Entity'nin tamamı yerine sadece bazı alanların çekilmesi.
DTO	Data Transfer Object. Katmanlar veya API arasında veri taşımak için kullanılan sade nesne.
Specification	Criteria API üzerine kurulu, birleştirilebilir sorgu koşulu.

Optimistic / Pessimistic lock	Çakışmayı sürümle (`@Version`) yakalamak / satırı kilitleyerek (`FOR UPDATE`) önlemek.
Index	Bir kolonda aramayı hızlandıran veritabanı yapısı.